

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Казанский (Приволжский) федеральный университет»

Институт вычислительной математики и информационных технологий
Кафедра прикладной математики и искусственного интеллекта

Отчёт

Реализация многослойного персептрона
для классификации данных

Работу выполнила:
студентка 4 курса группы 09-322
Назипова А.Н.

Казань 2026

Содержание

Введение	2
Постановка задачи	2
Структура проекта	3
Анализ данных	3
Общая информация о датасете	4
Описательная статистика	4
Визуальный анализ данных	5
Предобработка данных	7
Разделение данных	8
Стандартизация признаков	8
Математическая модель	8
Многослойный перцептрон	9
Функция активации Sigmoid	9
Прямое распространение	9
Функция Softmax	10
Функция потерь	10
Обратное распространение ошибки	11
Градиентный спуск	12
Алгоритм обучения	12
Выбор архитектуры	13
Инициализация весов	13
Batch size	14
Параметры обучения	14
Модульность программы	15
Результаты	15
График функции потерь	17
График точности	17
Результаты предсказания	18
Заключение	19

Введение

Данный проект посвящён практической реализации многослойного персептрона для решения задачи бинарной классификации. Основной целью работы является разработка нейронной сети без использования специализированных библиотек для машинного обучения, реализующей основные этапы обучения самостоятельно.

В качестве исходных данных используется набор данных, содержащий диагностические характеристики объектов. Каждый объект относится к одному из двух классов, обозначенных как B и M . Для каждого объекта представлены 30 числовых признаков.

Цель работы — реализовать полный цикл машинного обучения: от первичного анализа исходного датасета и его предварительной обработки до обучения многослойного персептрона, сохранения обученной модели и выполнения предсказаний на новых данных.

Особое внимание в работе уделяется реализации основных алгоритмов нейронной сети: прямого распространения сигнала, функции активации, функции Softmax, вычисления функции потерь, обратного распространения ошибки и градиентного спуска.

Также проводится визуальный анализ исходных данных, исследуется процесс обучения модели и оценивается качество классификации на обучающей и валидационной выборках.

Постановка задачи

Необходимо разработать программу классификации объектов по двум классам на основе 30 числовых признаков с использованием многослойного персептрона.

Нейронная сеть должна быть реализована самостоятельно с использованием базовых средств языка Python и библиотеки NumPy для выполнения матричных операций. Использование специализированных библиотек, реализующих нейронные сети, не применяется.

Структура проекта

Проект состоит из нескольких программ, каждая из которых отвечает за отдельный этап работы:

- `analysis.py` — первичный статистический и визуальный анализ данных;
- `split.py` — разделение исходного датасета на обучающую и валидационную выборки;
- `train.py` — реализация и обучение многослойного персептрона;
- `predict.py` — загрузка сохранённой модели и выполнение предсказаний;
- `data.csv` — исходный набор данных;
- `data_training.csv` — обучающая выборка;
- `data_validation.csv` — валидационная выборка;
- `saved_model.npy` — сохранённые параметры обученной модели.

Анализ данных

Перед обучением нейронной сети был проведён первичный анализ исходного набора данных. Анализ необходим для определения структуры датасета, проверки наличия пропущенных значений, оценки распределения классов и изучения распределения числовых признаков.

Общая информация о датасете

Исходный набор данных содержит 569 объектов и 32 столбца. Первый столбец содержит идентификатор объекта, второй столбец является целевой переменной, а остальные 30 столбцов содержат числовые признаки.

Таким образом, после исключения идентификатора и выделения целевой переменной для обучения модели используется 30 числовых признаков.

Целевая переменная принимает два значения - В и М.

Важной частью анализа является проверка пропущенных значений. Их отсутствие позволяет использовать исходные числовые признаки непосредственно перед стандартизацией без дополнительного заполнения пропусков.

Проверка распределения классов необходима для того, чтобы определить, насколько равномерно представлены различные варианты целевой переменной.

Описательная статистика

Для числовых признаков была рассчитана описательная статистика. Для каждого признака определялись количество значений, среднее значение, стандартное отклонение, минимальное значение, медиана и максимальное значение.

Основные характеристики нескольких признаков представлены в таблице 1.

Признак	Count	Mean	Std	Min	50%	Max
radius_mean	569	14.13	3.52	6.98	13.37	28.11
texture_mean	569	19.29	4.30	9.71	18.84	39.28
perimeter_mean	569	92.00	24.30	43.79	86.24	188.50
area_mean	569	654.89	351.91	143.50	551.10	2501.00
smoothness_mean	569	0.10	0.01	0.05	0.10	0.16
compactness_mean	569	0.10	0.05	0.02	0.09	0.35
concavity_mean	569	0.09	0.08	0.00	0.06	0.43
concave_points_mean	569	0.05	0.04	0.00	0.05	0.20

Таблица 1: Основные статистические характеристики признаков

Выводы:

1. Числовые признаки имеют различные масштабы, поэтому использование исходных значений без предварительной обработки может негативно повлиять на процесс обучения нейронной сети.
2. Признаки имеют различный разброс. Значения стандартного отклонения также существенно отличаются друг от друга.
3. Наличие различных диапазонов значений подтверждает необходимость стандартизации признаков перед подачей данных на вход нейронной сети.

Визуальный анализ данных

Для дополнительного анализа распределения числовых признаков были построены гистограммы. Первая группа гистограмм показывает распределение значений каждого из 30 признаков.

Гистограммы позволяют определить форму распределения признаков, наличие выраженной асимметрии и возможных экстремальных значений.

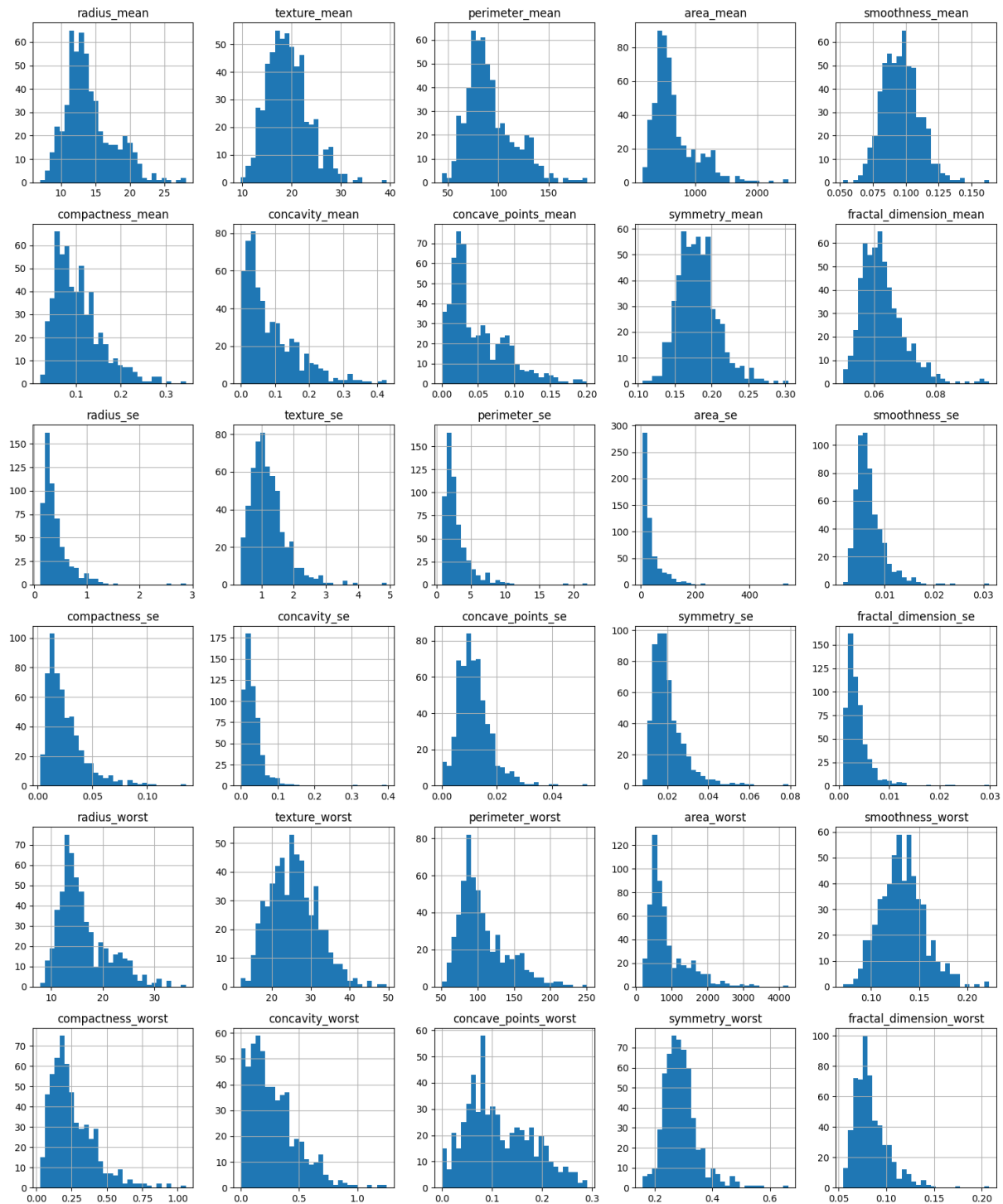


Рис. 1: Распределение значений всех 30 признаков

Для анализа различий между классами дополнительно были построены гистограммы, на которых одновременно отображаются распределения признаков для классов B и M .

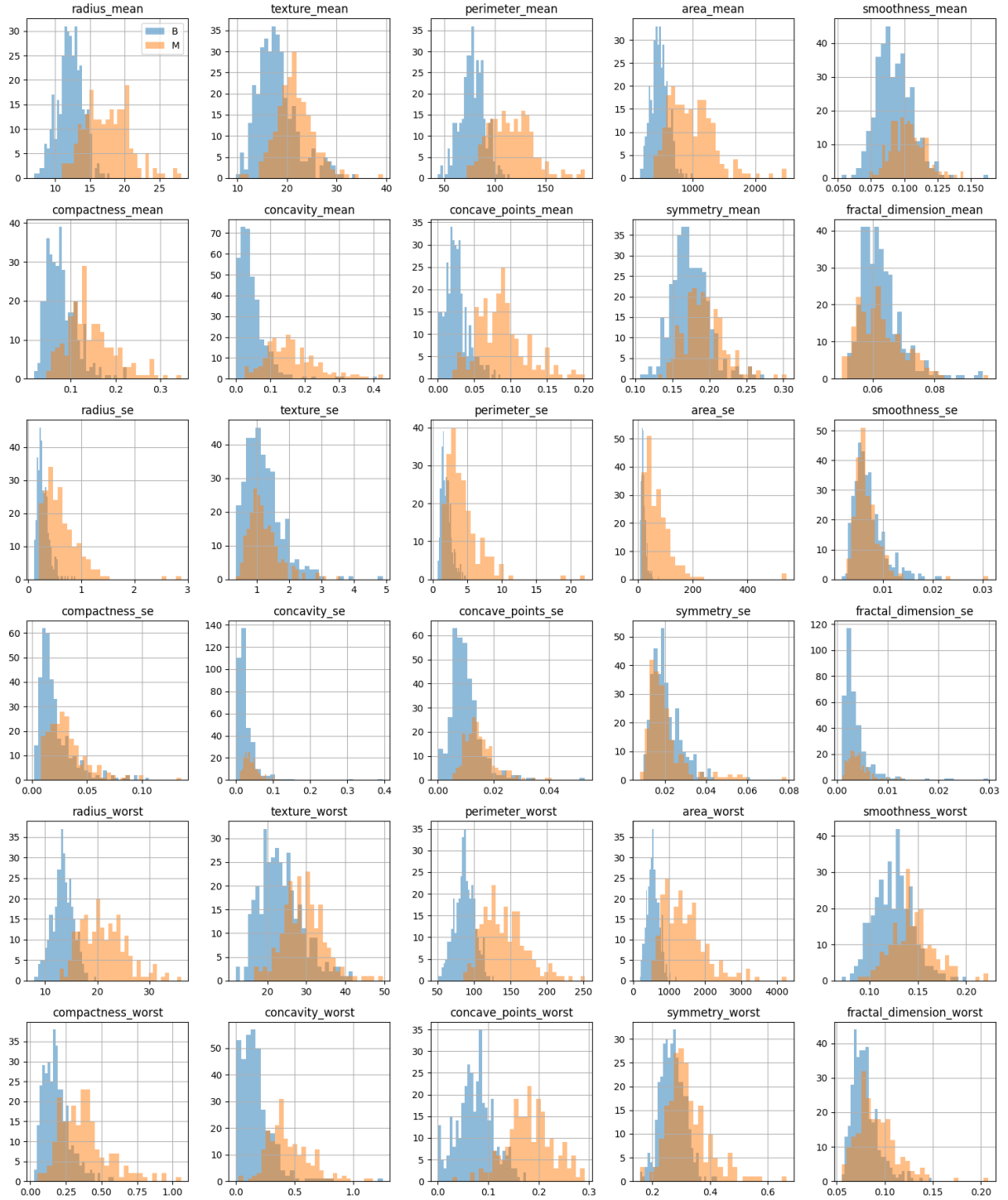


Рис. 2: Распределение признаков для классов В и М

Сравнение распределений показывает, что для ряда признаков центры и формы распределений классов *B* и *M* отличаются. Особенно заметны различия для признаков, связанных с размером и формой исследуемых объектов, например *radiusmean*, *perimetermean*, *areamean*, а также соответствующих им признаков группы *worst*. Это указывает на наличие зависимости между данными признаками и целевой переменной.

При этом распределения многих признаков частично перекрываются, поэтому использование только одного признака не позволяет надёжно разделить два класса.

На основании проведённого анализа было принято решение не исключать отдельные признаки на данном этапе, а использовать для обучения нейронной сети все 30 число-

вых признаков. Таким образом, входной слой многослойного персептрона содержит 30 нейронов, соответствующих всем числовым признакам исходного датасета. Перед подачей данных в сеть признаки стандартизируются, поскольку имеют существенно различные масштабы и диапазоны значений.

Предобработка данных

Разделение данных

Исходный датасет был разделён на две части:

- обучающую выборку — 454 объекта;
- валидационную выборку — 115 объектов.

Таким образом, примерно 80% данных используется для обучения модели и 20% — для проверки её качества.

Для воспроизводимости результата при случайном перемешивании данных используется фиксированное значение случайного зерна:

```
1 np.random.seed(42)
```

Использование фиксированного значения позволяет получать одинаковое разделение данных при повторном запуске программы.

Стандартизация признаков

Для приведения признаков к единому масштабу используется стандартизация по формуле Z-score:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (1)$$

где x — исходное значение признака, μ — среднее значение признака, а σ — стандартное отклонение.

Среднее значение и стандартное отклонение вычисляются только по обучающей выборке:

```
1 mean = X_train.mean(axis=0)
2 std = X_train.std(axis=0)
3
4 X_train = (X_train - mean) / (std + 1e-8)
5 X_valid = (X_valid - mean) / (std + 1e-8)
```

При этом параметры стандартизации обучающей выборки также используются для преобразования валидационной выборки, что позволяет избежать использования информации из валидационной выборки при обучении.

Так как на выходе нейронной сети используются два нейрона с функцией активации Softmax, каждый из них соответствует одному из двух классов целевой переменной: B и M . Значения на выходе Softmax интерпретируются как вероятности принадлежности объекта к каждому из классов. Сумма вероятностей двух выходных нейронов равна единице.

Для корректного сравнения предсказаний нейронной сети с истинными значениями целевая переменная представляется в виде one-hot вектора. В таком представлении для каждого объекта только один элемент вектора принимает значение 1, соответствующее правильному классу, а остальные элементы равны 0.

Для рассматриваемой задачи используются следующие варианты кодирования:

$$B = [1, 0] \quad (2)$$

$$M = [0, 1] \quad (3)$$

Математическая модель

Многослойный персептрон

Многослойный персептрон представляет собой нейронную сеть, состоящую из входного слоя, нескольких скрытых слоёв и выходного слоя.

Каждый нейрон получает значения предыдущего слоя, умножает их на соответствующие веса, складывает полученные значения и добавляет смещение:

$$z = XW + b \quad (4)$$

После этого к результату применяется функция активации:

$$a = f(z) \quad (5)$$

В используемой модели по умолчанию применяется архитектура:

$$30 \rightarrow 24 \rightarrow 24 \rightarrow 2 \quad (6)$$

где:

- 30 — количество входных признаков;
- 24 — количество нейронов первого скрытого слоя;
- 24 — количество нейронов второго скрытого слоя;
- 2 — количество нейронов выходного слоя.

Функция активации Sigmoid

На скрытых слоях используется сигмоидальная функция:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (7)$$

Значения сигмоиды находятся в диапазоне от 0 до 1. Ограничение значения аргумента с помощью `np.clip` позволяет избежать переполнения при вычислении экспоненты.

Прямое распространение

При прямом распространении входные данные последовательно проходят через все слои сети.

Для скрытого слоя сначала вычисляется линейная комбинация:

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)} \quad (8)$$

После этого применяется функция активации:

$$A^{(l)} = \sigma(Z^{(l)}) \quad (9)$$

На последнем слое вместо сигмоиды используется Softmax.

Реализация прямого распространения:

```

1 def forward(self, X):
2     activations = [X]
3
4     for i in range(len(self.weights) - 1):
5         z = activations[-1] @ self.weights[i] + self.biases[i]
6         a = sigmoid(z)
7         activations.append(a)
8
9     z = activations[-1] @ self.weights[-1] + self.biases[-1]
10    a = softmax(z)
11    activations.append(a)
12
13    return activations

```

Функция Softmax

На выходном слое используется функция Softmax. Её основная задача — преобразовать выходные значения сети в вероятностное распределение по классам.

Функция имеет вид:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (10)$$

где K — количество классов.

В данной задаче $K = 2$.

Сумма всех выходных вероятностей равна единице:

$$\sum_{i=1}^K p_i = 1 \quad (11)$$

Например, результат работы выходного слоя может иметь вид:

$$[0.93, 0.07] \quad (12)$$

Реализация Softmax:

```

1 def softmax(x):
2     x = x - np.max(x, axis=1, keepdims=True)
3     exp_x = np.exp(x)
4
5     return exp_x / np.sum(
6         exp_x,
7         axis=1,
8         keepdims=True
9     )

```

Вычитание максимального значения перед вычислением экспоненты используется для повышения численной устойчивости.

Функция потерь

Для оценки ошибки модели используется категориальная кросс-энтропия (Categorical Cross-Entropy):

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik}) \quad (13)$$

где:

- m — количество объектов;
- K — количество классов;
- y_{ik} — правильное значение для класса k ;
- $\hat{y}_{ik}$ — предсказанная вероятность класса k .

В реализации значения вероятностей дополнительно ограничиваются снизу небольшим числом, чтобы избежать вычисления $\log(0)$:

```

1 def cross_entropy(y_true, y_pred):
2     y_pred = np.clip(
3         y_pred,
4         1e-8,
5         1 - 1e-8
6     )
7
8     return -np.mean(
9         np.sum(
10            y_true * np.log(y_pred),
11            axis=1
12        )
13    )

```

Чем меньше значение функции потерь, тем ближе предсказанное распределение вероятностей к правильным ответам.

Обратное распространение ошибки

Для обучения нейронной сети необходимо определить, насколько изменение каждого веса влияет на итоговую ошибку.

Для комбинации Softmax и категориальной кросс-энтропии градиент по входу выходного слоя имеет простую форму:

$$\frac{\partial L}{\partial Z^{(l)}} = \frac{\hat{Y} - Y}{m} \quad (14)$$

где $\hat{Y}$ — предсказание сети, Y — правильные ответы, а m — размер текущего минипакета.

Градиент по весам слоя вычисляется как:

$$\frac{\partial L}{\partial W^{(l)}} = (A^{(l-1)})^T \frac{\partial L}{\partial Z^{(l)}} \quad (15)$$

Градиент по смещению:

$$\frac{\partial L}{\partial b^{(l)}} = \sum_i \frac{\partial L}{\partial Z_i^{(l)}} \quad (16)$$

Для скрытых слоёв используется производная сигмоиды:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (17)$$

Таким образом, ошибка распространяется от выходного слоя к входному, последовательно вычисляя необходимые градиенты.

Градиентный спуск

После вычисления градиентов параметры нейронной сети обновляются методом градиентного спуска:

$$W := W - \alpha \frac{\partial L}{\partial W} \quad (18)$$

$$b := b - \alpha \frac{\partial L}{\partial b} \quad (19)$$

где α — скорость обучения.

В программе обновление параметров выполняется следующим образом:

```
1 for i in range(len(self.weights)):
2     self.weights[i] -= (
3         learning_rate * gradients_w[i]
4     )
5
6     self.biases[i] -= (
7         learning_rate * gradients_b[i]
8     )
```

Алгоритм обучения

Выбор архитектуры

По умолчанию используется следующая архитектура:

$$30 \rightarrow 24 \rightarrow 24 \rightarrow 2 \quad (20)$$

Первый слой получает 30 числовых признаков.

Первый скрытый слой содержит 24 нейрона. Второй скрытый слой также содержит 24 нейрона. На выходном слое расположены два нейрона, соответствующие двум классам.

Архитектура формируется программно, что позволяет изменять количество нейронов в скрытых слоях с помощью аргументов командной строки.:

```
1 layer_sizes = [X_train.shape[1]] + args.layer + [2]
2
3 print(f"Architecture : {layer_sizes}")
4
5 model = MLP(layer_sizes)
```

Инициализация весов

Перед началом обучения веса нейронной сети необходимо инициализировать.

Для начальной инициализации весов использовалось случайное равномерное распределение. Граница диапазона определяется формулой

$$limit = \sqrt{\frac{6}{n_{in}}}, \quad (21)$$

где n_{in} — количество входных нейронов текущего слоя.

После вычисления значения $limit$ каждый вес выбирается случайным образом из интервала

$$W_{ij} \in [-limit, limit]. \quad (22)$$

Такой способ позволяет начать обучение со случайных, но ограниченных по величине весов. Ограничение диапазона необходимо для того, чтобы начальные значения сигналов в нейронной сети не становились слишком большими. Таким образом, величина начальных весов зависит от количества входных нейронов: чем больше n_{in} , тем уже выбираемый диапазон. Это позволяет учитывать размер входных данных при инициализации параметров каждого слоя.

После этого значения весов выбираются из равномерного распределения:

```
1 limit = np.sqrt(6 / n_in)
2
3 weights = np.random.uniform(
4     -limit,
5     limit,
6     (n_in, n_out)
7 )
8
9 biases = np.zeros((1, n_out))
```

Смещения первоначально устанавливаются равными нулю.

Batch size

Обновление весов выполняется не после обработки всей обучающей выборки сразу, а небольшими группами объектов.

В качестве размера "пакета" используется значение:

$$batch_size = 8 \quad (23)$$

Перед каждой эпохой обучающая выборка случайно перемешивается:

```
1 indices = np.random.permutation(len(X_train))
2
3 X_shuffled = X_train[indices]
4 y_shuffled = y_train[indices]
```

После этого данные последовательно разбиваются:

```
1 for i in range(0, len(X_shuffled), args.batch_size):
2
3     X_batch = X_shuffled[
4         i:i + args.batch_size
5     ]
6
7     y_batch = y_shuffled[
8         i:i + args.batch_size
9     ]
10
11     activations = model.forward(X_batch)
12
13     model.backward(
14         activations,
15         y_batch,
16         args.learning_rate
17     )
```

Такой подход позволяет выполнять несколько обновлений весов за одну эпоху.

Параметры обучения

В итоговом эксперименте использовались следующие параметры:

Параметр	Значение
Количество скрытых слоёв	2
Нейроны скрытого слоя 1	24
Нейроны скрытого слоя 2	24
Выходных нейронов	2
Количество эпох	55
Размер батча	8
Скорость обучения	0.03
Random seed	42

Таблица 2: Параметры обучения многослойного персептрона

Скорость обучения и количество эпох были выбраны экспериментальным путём на основе анализа результатов нескольких запусков нейронной сети с различными параметрами. В итоговом эксперименте было выбрано значение скорости обучения 0.03.

Количество эпох также определялось экспериментально. В качестве итогового значения было выбрано 55 эпох. К этому моменту функция потерь на обучающей и валидационной выборках достигла низких значений, а точность классификации составила 98.24

Таким образом, параметры обучения были выбраны на основании сравнения экспериментальных результатов. Используемые значения обеспечивают высокую точность классификации при небольших значениях функции потерь на обеих выборках.

Модульность программы

Одним из требований проекта является возможность изменения архитектуры нейронной сети.

Для этого параметры обучения задаются через аргументы командной строки:

```
1 parser.add_argument(  
2     "--layer",  
3     type=int,  
4     nargs="+",  
5     default=[24, 24]  
6 )  
7  
8 parser.add_argument(  
9     "--epochs",  
10    type=int,  
11    default=55  
12 )  
13  
14 parser.add_argument(  
15     "--batch_size",  
16     type=int,  
17     default=8  
18 )  
19  
20 parser.add_argument(  
21     "--learning_rate",  
22     type=float,  
23     default=0.0314  
24 )
```

Например, архитектуру можно изменить следующим образом:

```
1 python train.py --layer 32 16 --epochs 60
```

В этом случае будет создана сеть:

$$30 \rightarrow 32 \rightarrow 16 \rightarrow 2 \quad (24)$$

Таким образом, архитектура не является жёстко заданной непосредственно в классе модели.

Результаты

В процессе обучения после каждой эпохи вычислялись значения функции потерь и точности как на обучающей, так и на валидационной выборках.

Для оценки качества использовались следующие показатели:

$$Accuracy = \frac{N_{\text{correct}}}{N_{\text{total}}} \quad (25)$$

где N_{correct} — количество правильно классифицированных объектов, а N_{total} — общее количество объектов.

Для каждой эпохи выводились четыре значения:

- Training Loss — функция потерь на обучающей выборке;
- Validation Loss — функция потерь на валидационной выборке;
- Training Accuracy — точность на обучающей выборке;
- Validation Accuracy — точность на валидационной выборке.

Фрагмент вывода программы:

```
1 epoch 49/55 - loss: 0.0607 - val_loss: 0.0689
2 - acc: 0.9846 - val_acc: 0.9913
3
4 epoch 50/55 - loss: 0.0605 - val_loss: 0.0681
5 - acc: 0.9846 - val_acc: 0.9913
6
7 epoch 51/55 - loss: 0.0599 - val_loss: 0.0685
8 - acc: 0.9846 - val_acc: 0.9913
9
10 epoch 52/55 - loss: 0.0595 - val_loss: 0.0684
11 - acc: 0.9846 - val_acc: 0.9913
12
13 epoch 53/55 - loss: 0.0592 - val_loss: 0.0682
14 - acc: 0.9846 - val_acc: 0.9913
15
16 epoch 54/55 - loss: 0.0587 - val_loss: 0.0682
17 - acc: 0.9846 - val_acc: 0.9913
18
19 epoch 55/55 - loss: 0.0588 - val_loss: 0.0679
20 - acc: 0.9824 - val_acc: 0.9913
```

Итоговые показатели обучения представлены в таблице 3.

Показатель	Значение
Training Loss	0.0588
Validation Loss	0.0679
Training Accuracy	98.24%
Validation Accuracy	99.13%

Таблица 3: Итоговые показатели обучения модели

Полученные результаты показывают, что модель успешно обучилась классифицировать объекты двух классов.

Точность на валидационной выборке составляет 99.13%, что соответствует 114 правильно классифицированным объектам из 115.

График функции потерь

Для визуального контроля процесса обучения был построен график изменения функции потерь на обучающей и валидационной выборках.

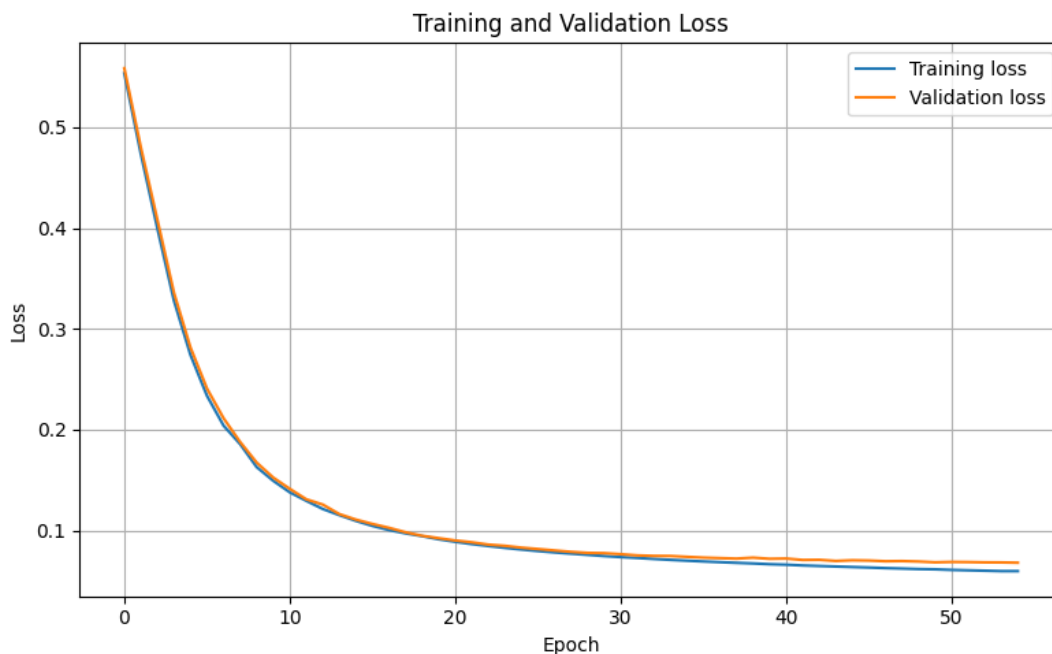


Рис. 3: Изменение функции потерь в процессе обучения

По графику можно оценить скорость сходимости модели. В процессе обучения значение функции потерь уменьшается, что говорит об успешной настройке весов нейронной сети.

К концу обучения значения Training Loss и Validation Loss остаются достаточно близкими друг к другу. Это свидетельствует об отсутствии выраженного переобучения модели.

График точности

Также был построен график изменения точности классификации:

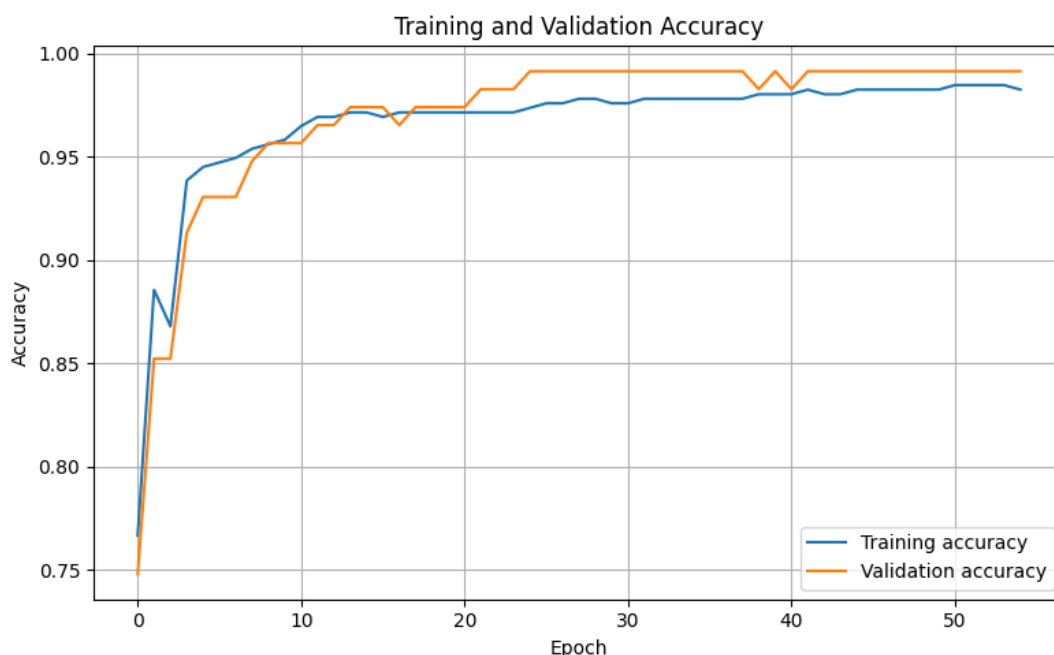


Рис. 4: Изменение точности в процессе обучения

Точность на обучающей выборке постепенно увеличивается в процессе обучения. Валидационная точность достигает значения около 99%.

Небольшие изменения точности между соседними эпохами связаны с использованием случайного перемешивания данных и мини-пакетного обучения.

Результаты предсказания

После завершения обучения сохранённая модель была загружена программой `predict.py` и использована для классификации объектов валидационной выборки, содержащей 115 объектов. Для оценки качества работы модели рассчитывались количество правильных и ошибочных предсказаний, точность классификации (Accuracy) и значение категориальной кросс-энтропии (Cross-Entropy Loss).

В итоговом эксперименте использовалась архитектура многослойного персептрона $30 \rightarrow 24 \rightarrow 24 \rightarrow 2$ с двумя скрытыми слоями. После 55 эпох обучения были получены следующие результаты.

Показатель	Значение
Размер набора данных	115
Правильных предсказаний	114
Ошибочных предсказаний	1
Cross-Entropy Loss	0.0680
Accuracy	99.13%

Таблица 4: Результаты классификации валидационной выборки

Полученная точность классификации составила 99.13%: из 115 объектов модель правильно определила класс 114 объектов и допустила одну ошибку. Значение Cross-Entropy Loss, равное 0.0680, свидетельствует о небольшом расхождении между предсказанными вероятностями и истинными метками классов.

Валидационная выборка не использовалась для обновления весов нейронной сети в процессе обучения, поэтому полученный результат позволяет оценить способность модели классифицировать объекты, не участвовавших в настройке её параметров. Высокая точность на одной выборке не гарантирует такого же качества на любых новых данных, однако результаты эксперимента показывают, что реализованный многослойный персептрон успешно справляется с поставленной задачей бинарной классификации.

Заключение

В ходе работы был реализован многослойный персептрон для решения задачи бинарной классификации.

Был выполнен полный цикл обработки данных: проведён первичный статистический и визуальный анализ исходного датасета, выполнено разделение данных на обучающую и валидационную выборки, проведена стандартизация числовых признаков и преобразование целевой переменной в one-hot представление.

В рамках проекта самостоятельно реализованы основные компоненты нейронной сети:

- многослойная архитектура нейронной сети;
- инициализация весов и смещений;
- прямое распространение;
- сигмоидальная функция активации;
- функция Softmax на выходном слое;
- категориальная кросс-энтропия;
- обратное распространение ошибки;
- вычисление градиентов;
- градиентный спуск.

Реализованная архитектура по умолчанию имеет вид

$$30 \rightarrow 24 \rightarrow 24 \rightarrow 2 \quad (26)$$

и содержит два скрытых слоя.

Одним из важных результатов работы является возможность конфигурирования архитектуры сети через аргументы командной строки. Это позволяет изменять количество скрытых слоёв и количество нейронов без изменения основной реализации класса многослойного персептрона.

Для контроля процесса обучения были реализованы вывод значений функции потерь и точности на каждой эпохе, а также построение двух графиков: изменения функции потерь и изменения точности на обучающей и валидационной выборках.

Разработанная нейронная сеть успешно решает поставленную задачу классификации. Реализация основных алгоритмов выполнена самостоятельно без использования специализированных библиотек нейронных сетей, что позволяет проследить весь процесс обучения многослойного персептрона от подачи исходных данных до получения итогового предсказания.